

University of São Paulo
Institute of Mathematics and Statistics
Computer Science Department

—

*Research Project for the Doctoral Fellowship
São Paulo Research Foundation (FAPESP)*

Mapping and Mitigating Bottlenecks in the Linux Kernel Development Model

Student: David Tadokoro

Advisor: Paulo Meirelles

Research Internship Abroad advisor: Gregorio Robles (URJC)

davidbtadokoro@usp.br, paulormm@ime.usp.br, grex@gsyc.urjc.es

Abstract

The Linux kernel, a FLOSS project with over 30 years of active development, is central to modern computing. Combined with system tools and utilities from the GNU Project, it forms the GNU/Linux ecosystem, which is fundamental for services in our society, such as the Internet, and ubiquitous in the bleeding edge of fields like Machine Learning. The development of Linux is based on the collaboration of thousands of developers worldwide who interact primarily through email and mailing lists. It is a unique development model involving many processes and practices (workflows) that extend beyond the technical aspects of software development. Despite its success, Linux relies on critical workflows and key personnel, raising concerns about sustainability and workforce renewal. As the codebase and community grow in size and complexity, the entry barrier for newcomers and the workload per maintainer should not increase simultaneously. More interestingly, significant time and resources are spent developing tools that address limitations in the Linux kernel development model that the practitioners feel in their day-to-day work, yet these challenges are underexplored in academia. This research project aims to: 1) propose taxonomies that accurately detail the workflows that define the Linux kernel development model and identify inherent bottlenecks; 2) mitigate bottlenecks by developing FLOSS tools that support the Linux ecosystem.

Keywords: Linux, FLOSS, Linux kernel development model, FLOSS development model, bottleneck mitigation, software engineering.

1. Introduction

Computers, ranging from personal devices to embedded systems in automobiles, are essential in modern life and typically consist of hardware (physical components) and software (programs controlling the hardware). A key software component is the Operating System (OS), which acts as a bridge between hardware and software, with its kernel being the core program managing hardware details and enabling other programs to function [17]. The Linux kernel, a Free/Libre/Open Source Software (FLOSS), exemplifies this by being highly customizable and widely used across various sectors, including Internet infrastructure and machine learning, making it a critical solution for real-world problems. Its adaptability and lack of licensing costs have made it central to the technology industry, supported by diverse stakeholders.

1.1. The Linux kernel FLOSS ecosystem

The Linux kernel project is structured into interconnected subsystems, analogous to neighborhoods in a city [19], each managed by maintainers who oversee contributions from developers. The `MAINTAINERS` file lists maintainers and their respective codebase portions. However, entries in the `MAINTAINERS` file often overlap and share code, forming clusters rather than isolated entities that more closely represent subsystems [1]. The Linux community operates in a decentralized, global, and asynchronous manner, reflecting its distributed nature. Figure 1a illustrates this complexity, showing an undirected graph of `MAINTAINERS` entries and their codebase overlaps, highlighting the interconnection of subsystems.

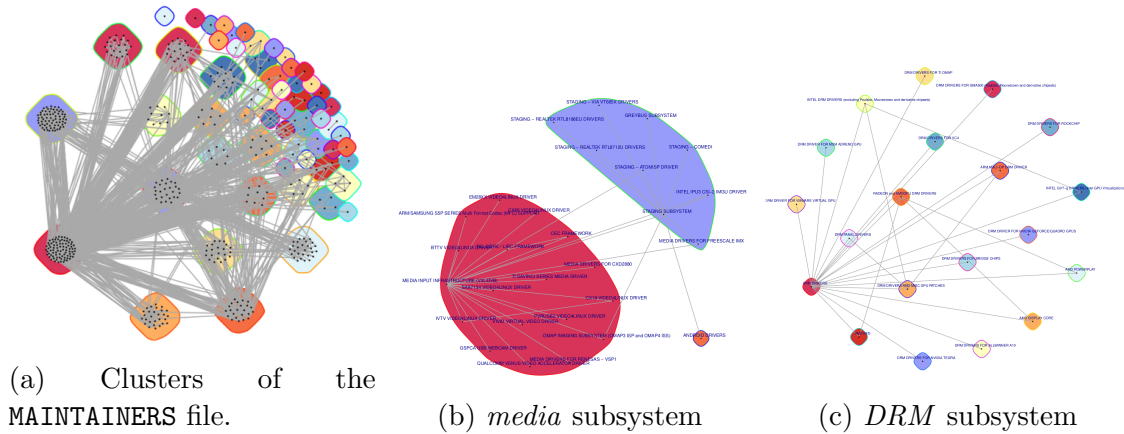


Figure 1: Subsystems of the Linux kernel. *Source* [1].

Despite their coupling, each subsystem has unique governance, maintainership models, and domain-specific characteristics. Figures 1b and 1c provide a closer look at the *media* and *DRM* subsystems, respectively, showcasing their distinct organizational structures. These examples underscore the diversity within the Linux kernel subsystems, even as they remain deeply interconnected [19].

1.1.1. How the Linux kernel subsystems connect

As mentioned, the Linux kernel project operates like a city divided into neighborhoods (subsystems), each governed by maintainers and supported by their ecosystems. These subsystems manage specific codebase portions within a hierarchical *web of trust*, where upper levels delegate tasks to lower levels. Linus Torvalds, the creator of Linux, sits at the top as a *benevolent dictator* [2], resolving disputes and overseeing the project. Despite this hierarchy, the structure is flatter than it seems, with most maintainers submitting pull requests¹ directly to Linus, bypassing mid-level maintainers [3]. While scalable, this model has led to inefficiencies, such as 44% of contributions being ignored over the past decade [16], causing frustration and wasted effort within the community.

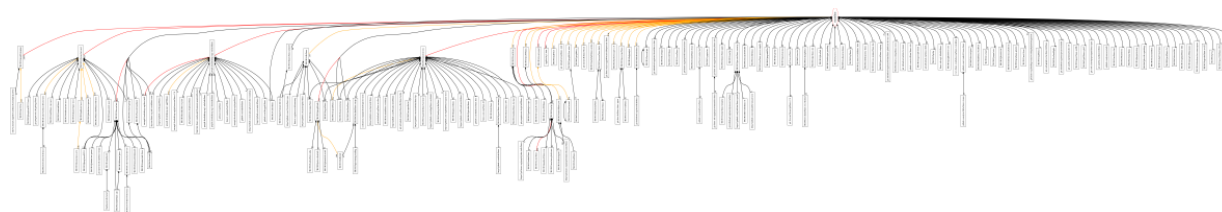


Figure 2: Flow of changes from the Linux 4.4 development cycle as a tree plot showing the flat hierarchy of the development model (this figure is not supposed to be readable). *Source:* [2].

Subsystems function as independent projects with their own forks, synchronizing with the *mainline* (Linus’ tree) via pull requests. This decentralized model offers flexibility but creates dependencies on key maintainers, particularly Linus. Figure 2 depicts the flat hierarchy of the Linux 4.4 development cycle, emphasizing Linus’ central role and the ecosystem interconnection. Although this structure has driven Linux success, it exposes vulnerabilities, such as maintainer overload and reliance on critical individuals, which could threaten the project long-term sustainability.

¹A request to merge changes made in one fork of a project into the original repository. Not to be confused with *GitHub Pull-Requests*.

1.1.2. Patches and the Linux kernel development process

At the heart of the Linux kernel development process is the *patch*, a text document that describes changes between two versions of a source tree and represents a contribution to the project [19]. Similar to a git commit, a patch includes a code differential and a descriptive message but is formatted for email submission via mailing lists. Development, bug reporting, and discussions occur on these lists, with contributors submitting patches for review. Maintainers and the community provide feedback, leading to iterative revisions in a process known as the *patch-reviewing process*. This back-and-forth ensures high-quality contributions [14], and once a patchset (set of patches that comprise a contribution) is deemed ready, it begins its journey through the upstream (upper levels of Linux hierarchy) repositories.

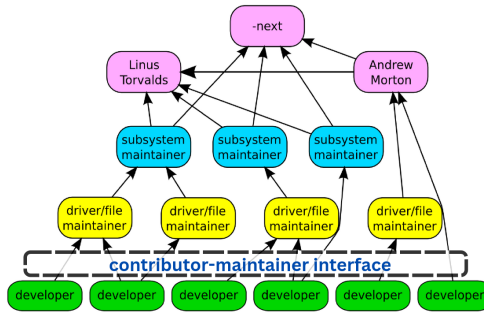


Figure 3: Patches flow into the *mainline*. *Source*: [11].

Patches are first merged into the subsystem official repository, where maintainers have jurisdiction, and then progress up the hierarchy until they reach Linus Torvalds in the *mainline* repository. Figure 3 illustrates this flow, showing how patches move from contributors to the mainline. Along the way, *Continuous Integration* (CI) practices, such as stabilization in *-next* branches, ensure changes do not destabilize the project. For example, the *AMD DRM* subsystem uses the *amd-staging-drm-next* branch for this purpose.

Despite its effectiveness, the patch and email-based development process is considered outdated. *Izquierdo-Cortazar et al. (2017)* [10] argue for modernizing the review process, as mailing lists require proactive subscription and often face delivery issues. Tools like the Lore kernel archives and *GitGitGadget* aim to address these challenges by providing real-time web access to mailing lists and enabling contributions via *GitHub Pull-Requests*, respectively. These efforts highlight the need to adapt the Linux development model to contemporary standards while maintaining its collaborative and decentralized nature.

1.1.3. The Linux kernel development model

The Linux kernel development model encompasses a variety of *workflows* that extend beyond technical functionalities, including practices like submitting patches to mailing lists, adhering to subsystem-specific rules, and following strict development cycles. These workflows are essential for maintaining the project structure and quality but also reveal bottlenecks that hinder efficiency. As noted by *Corbet and Kroah-Hartman (2017)* [4], tools are critical for managing the complexity of a project as large as Linux. For instance, git, created by Linus Torvalds in 2005, replaced the previous version control system and has since become indispensable for developers worldwide.

To address these bottlenecks, the Linux community develops tools ranging from official scripts like `get_maintainers.pl` (finds patch recipients) to standalone projects like `b4` (fetches patches from archived mailing lists). Many developers also create personal scripts to solve specific issues, leading to duplicated efforts and a lack of systematic cataloging of solutions. Researchers involved in this project are actively developing FLOSS tools, such as the *kworkflow*² project and its subproject *patch-hub*. These tools aim to unify solutions under a single interface, reducing inefficiencies and improving the overall development experience.

1.2. Problem outline

On one hand, the Linux kernel project steadily grows in size, complexity, and number of developers involved. On the other hand, we can name a few of the concerns about its development model detected by academia and practitioners: (i) Scalability of the community and the workflows; (ii) Renewal of the workforce; (iii) Key personnel that occupy critical roles and are often irreplaceable; (iv) Waste of community effort and resources; (v) Inefficient and outdated technologies that are at the core of the model; (vi) *Ad-hoc* solutions to workflow bottlenecks and duplication of work.

More severely, academic studies on the Linux project mostly detach from observing the practice and lack a proper investigation of these matters, carrying homogeneous and biased views [19]. As a result, there are misleading conclusions from the academic that surface when contrasted with the perspective of the Linux community (*Grey Literature*) and with the close observation of the practice (*ethnographic case-studies*) [19]. We also point out that the ubiquity of *ad-hoc* solutions to bottlenecks general to most Linux developers

²<https://kworkflow.org>

has an impact on the scalability of workflows and the workforce and compromises the project sustainability in the long term; therefore, in-depth academic research into the Linux ecosystem that unifies state-of-the-art and state-of-the-practice can lead to informed decisions on the development of critical tools that support the Linux umbrella project.

2. Objectives

Given the gap in the academic understanding of the Linux kernel development model when contrasted with the practice, there are undoubtedly several topics that are either not covered at all or investigated in a shallow manner by academic literature. For instance, in the Linux community, there is a quasi-mature discussion about the social challenges, and actions are taken in that regard, like a self-imposed break by Linus Torvalds [18] after strong disagreements with the community. There is even a *Code of Conduct* (CoC)³ that has a committee to handle reported misconduct. Nevertheless, no academic studies covered these topics at all [19].

Even though social challenges are significant - after all, software engineering considers the process of developing software as a socio-technical phenomenon - this proposed project focuses on understanding the overarching workflows (i.e., the processes and practices) that define the Linux kernel development model. In this sense, we present the two primary objectives of this work:

- **Objective 1:** Validate a theoretical framework that accurately describes the overarching workflows of the Linux kernel development model and allows us to identify bottlenecks;
- **Objective 2:** Develop FLOSS tools that efficiently mitigate bottlenecks in the Linux kernel development model using informed decisions based on a validated theoretical framework.

We acknowledge that successfully reaching these objectives does not directly solve all the previously outlined problems. Nonetheless, it does serve as a starting point for answering them. To fulfill these objectives, we will adopt a multi-method approach focusing on *ethographic case-studies* to be close to the practice while also considering the academic literature, grey literature, and community perspectives to triangulate findings and fortify their accuracy. To guide the research, we present three questions:

³[Permalink to latest Code of Conduct file.](#)

1. **RQ.1:** What overall workflows compose the Linux kernel development model?
2. **RQ.2:** Which processes and/or practices generate bottlenecks in the Linux kernel development model that compromise its efficiency and scalability?
3. **RQ.3:** Which bottlenecks can be mitigated by streamlining processes through automation?

2.1. Expected and preliminary results

In **Objective 1**, we aim to validate a taxonomy within the framework of *Grounded Theory* (GT), which involves constructing a structured classification system based on empirical data [8] and its use has become commonplace in the software engineering field [12]. The produced taxonomy will categorize the macro workflows in Linux kernel development, potentially including nested subcategories for finer-grained analysis. Over time, this taxonomy is expected to evolve into an *emergent theory*, a coherent explanatory framework that extends beyond mere classification. This emergent theory will provide a mature and solid foundation for understanding and deriving conclusions about the Linux kernel development model, offering insights into its complexities and workflows.

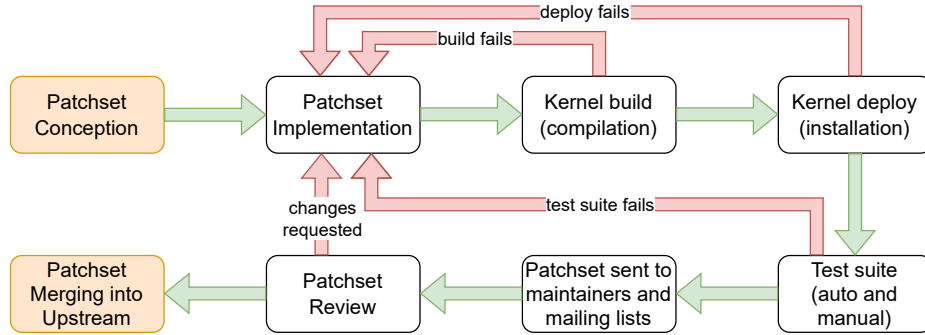


Figure 4: Concept map of the patchset-development workflow.

Figures 4 and 5 illustrate two key workflows in Linux development. Figure 4 depicts the patchset-development workflow, which spans from the conception of a contribution to its submission, review, and start of merging into the upstream. It includes technical processes like kernel compilation and deployment, as well as practices such as sending patches to the correct maintainers and participating in the review process. Figure 5 focuses on the patchset-reviewing workflow, which is more relevant to maintainers. It outlines steps from fetching and selecting patchsets to providing feedback or initiating the merging process.

Both workflows highlight the interactive and collaborative nature of Linux development, with variations depending on the subsystem and development context. These concept maps were derived from *Participant Observation*, where the doctoral student: (i) contributed to subsystems like *AMD GPU*, *Industrial I/O*, and *staging*; (ii) developed and maintained tools that support the community; (iii) mentored newcomers into Linux development.

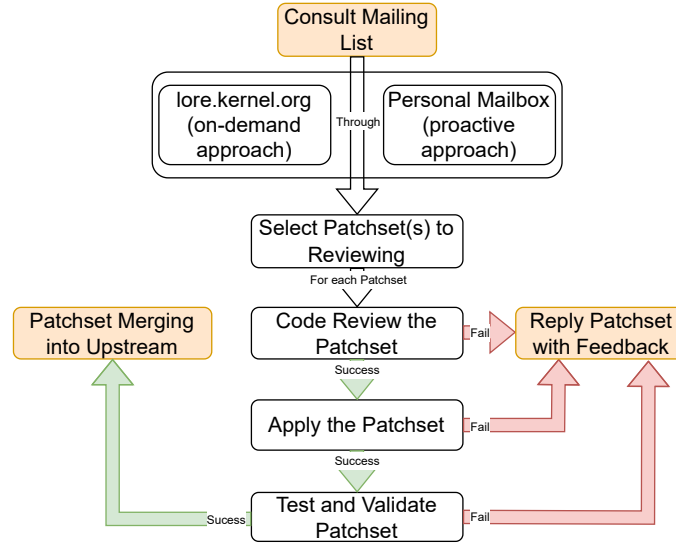


Figure 5: concept map of the patchset-reviewing workflow.

The doctoral student’s ongoing development of FLOSS tools to support the Linux ecosystem aligns with **Objective 2**. By leveraging insights from the taxonomy iterations, informed decisions can be made to address bottlenecks and improve workflows. These tools, such as those aiding patch submission and review, aim to streamline processes and enhance efficiency within the Linux development model. This iterative approach ensures that the tools are grounded in the realities of the Linux ecosystem, addressing actual needs identified through systematic analysis.

3. Justification

Our research group has extensive experience with the Linux ecosystem, including current and past members who have actively contributed to subsystems such as *AMD GPU*, *Industrial I/O (IIO)*, and *Virtual Kernel Mode Setting (VKMS)*. Many of these members work professionally as contributors and maintainers, dedicating themselves entirely to Linux development. For instance, our students accounted for 20% of all accepted contribu-

tions to the IIO subsystem during a 90-day period in 2019. Drawing from this experience, our group developed a FLOSS project called *kworkflow* (*kw*), a *Developer Automation Workflow System* (DAWS) designed to streamline and automate workflows for Linux developers.

*kworkflow*⁴ provides a unified interface offering solutions to common kernel development challenges, such as abstracting hardware and distribution-specific complexities for tasks like configuring, compiling, and installing custom kernels. It also includes tools for submitting contributions and managing reviews, which have gained traction among industry developers. In 2023, the doctoral student became a maintainer of *kworkflow* and the primary maintainer of its subproject, *patch-hub*⁵. This *Terminal User Interface* (TUI), designed for maintainers, simplifies interaction with contributions sent to mailing lists and review management, addressing one of the most critical bottlenecks in the development model.

Our firsthand experience and continuous interaction with researchers and practitioners have generated valuable insights. By systematically mapping Linux kernel development workflows and identifying bottlenecks, we aim to bridge the gap between theory and practice, opening avenues for future research and potentially applying these findings to other software engineering contexts. Pragmatically addressing these bottlenecks with high-quality tools will benefit the Linux ecosystem - contributors, maintainers, testers, and other stakeholders - by streamlining complex processes, enhancing productivity, reducing workload, increasing contribution throughput, and scaling the workforce. Ensuring these tools are FLOSS is crucial, as it allows the Linux community to provide feedback and contribute organically, aligning with **Objective 2**.

Additionally, addressing these challenges could free up resources to tackle underprioritized issues, such as the project environmental sustainability. Given Linux pervasive role in technology - from embedded systems and Internet infrastructure to supercomputers and AI machine learning model training - making it more environmentally friendly could have a significant global impact. Furthermore, improving Linux quality as a software solution could create a positive ripple effect across modern computing, reinforcing its importance as a cornerstone of FLOSS development.

⁴[kworkflow site](#) and [kworkflow repository](#).

⁵[patch-hub repository](#).

3.1. Evidence about the gap in academic understandings of the Linux kernel development model

A symptom of the misleading conclusions from the scientific community about the Linux kernel development model is the divergent definitions given by different authors.

For *Rigby et al. (2014)* [16], the model is a dictatorship that relegates newcomers and has a rigid and subservient culture; For *Fogel (2017)* [6], the “forkability” of the project results in a “benevolent dictator” model where Linus does not have full authority on the project; *Lindberg et al. (2014)* [13], believes that a core group of developers takes most of the reviews and is also exclusively responsible for approving code changes, following a *Cathedral model* [15]; finally, *Shaikh and Henfridsson (2017)* [9] argue that four governance types coexist in the Linux kernel development: autocracy, oligarchy, federation, meritocracy.

The lack of a consensus on a primordial aspect, such as categorizing the Linux kernel development model, indicates how academic investigations are at an early stage. At last, *Wen (2021)* [19], using the qualitative research method of *Directed Content Analysis*, produced mind maps that describe the Linux kernel development model. They showed, at large, gaps and misconceptions from the scientific community when crossed with community perspectives.

3.2. Empirical evidence about the unsustainability of the Linux kernel development model

We have produced empirical quantitative evidence indicating that the Linux kernel development model may be unsustainable due to its current structure⁶. Figures 6, 7, and 8 are plots that show statistics about the development of Linux over the last 10 years. Due to the periodicity of releases that follow subsequent six to eight-week cycles, we can use the Linux versions as consistent timestamps to analyze the evolution of the project. In this sense, the aforementioned figures have the Linux versions in the abscissa from 3.19 (launched on 8 February 2015) until 6.13 (launched on 20 January 2025).

Figure 6 displays the increase of *physical Lines of Code* (physical LOC)⁷ and shows how the Linux kernel codebase is steadily growing regarding its size. We can derive that

⁶These experiments are reproducible with *Docker* following the instructions in [this repository](#).

⁷A software metric that measures the number of source lines of code (excluding blank ones and comments) in a program source code.

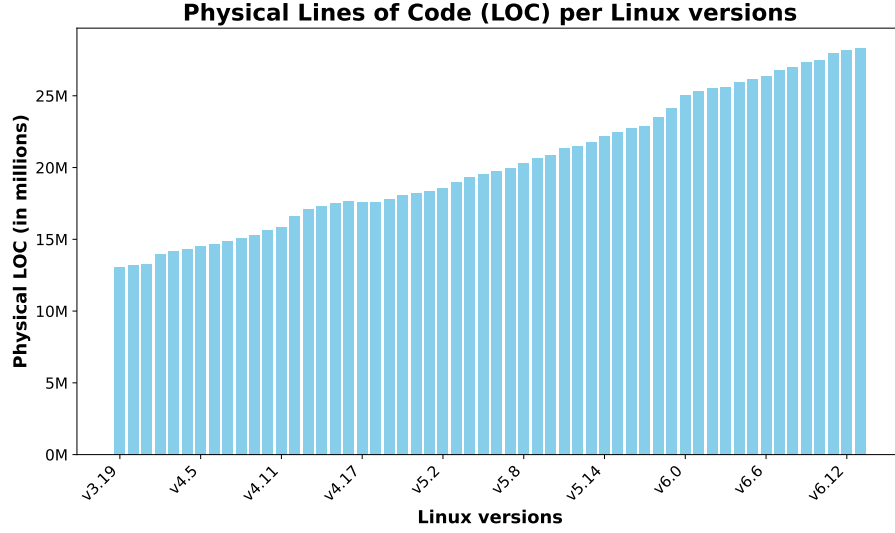


Figure 6: Linux physical LOC from version 3.19 to 6.13.

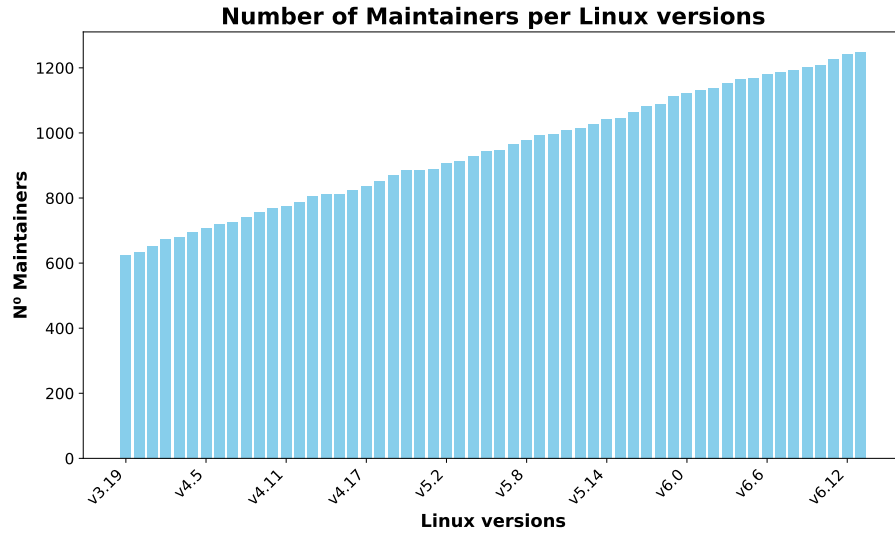


Figure 7: Number of Linux maintainers from version 3.19 to 6.13.

the project complexity is also growing as more features, drivers, and the like are continuously being merged. Interestingly, by investigating the entries in the `MAINTAINERS` file, we produced Figure 7 that depicts how the number of maintainers steadily increases.

Notwithstanding the increase in the number of maintainers, we can see in Figure 8 that the amount of commits in each development cycle of Linux fluctuates between 13000 and 17500 since version 3.19, excluding outliers. Every commit is different as an individual commit can be a massive change, while a set of ten commits can be a single line change ten

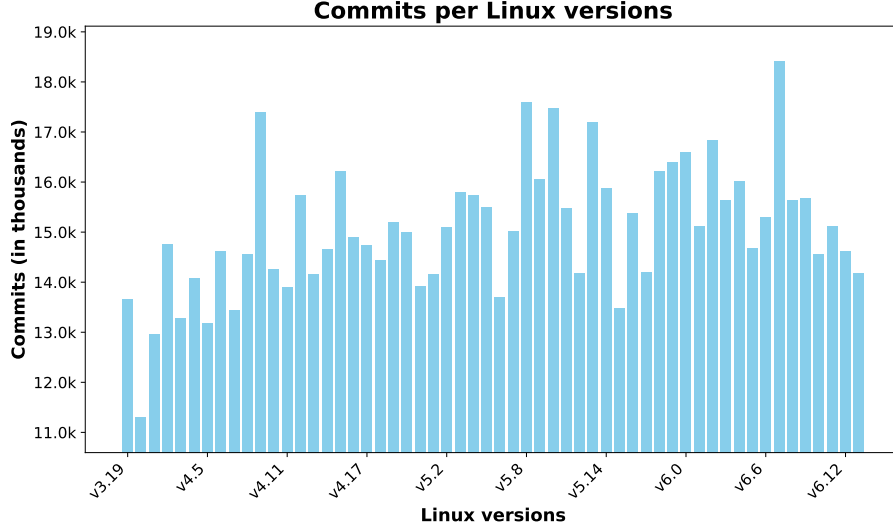


Figure 8: Commits in the development cycle of versions 3.19 to 6.13.

times, and a difference of four and a half thousand commits is expressive, so we can not derive solid conclusions based on these statistics. However, just the fact that the number of commits has fluctuated consistently indicates that the contribution throughput does not scale in the same fashion as the number of maintainers.

The Linux kernel development model functions as a self-sustaining and highly specialized ecosystem. However, by extrapolating the empirical evidence, we can assume that the model reliance on a continuous influx of skilled contributors, a highly dedicated pool of maintainers, and ever-increasing code complexity suggests characteristics similar to an “economic bubble” where short-term growth may mask long-term sustainability risks.

4. Research Methods

For this proposed research, we envision a multi-method approach that is a combination of these two research methods: (i) Ethnographic Case-Studies (Participant and Non-Participant Observation) and (ii) Multivocal Literature Review.

4.1. Ethnographic Case-Studies

An ethnographic case study is a qualitative research approach that combines an in-depth, context-rich investigation of a specific case with ethnographic methods such as participant and non-participant observation, interviews, and document analysis. It seeks

to understand social practices, cultural meanings, and lived experiences within a particular setting, emphasizing the relations between individual agency and broader structures [5].

As mentioned, the researchers have a vast background in academia and practice with the Linux ecosystem, firsthand or inherited from the experience of other researchers and people involved in the ecosystem. These two sources of insights are what we call *Participant Observation* and *Non-Participant Observation*, respectively, and together represent the primary research method of this project.

4.1.1. Participant Observation

In regards to the former, the aim of the participant observation is for the doctoral student to continue his involvement in the Linux project in subsystems like the AMD GPU but go deeper in terms of other subprojects (*Rust for Linux*⁸ is one marked to be explored) and in terms of the impact of contributions and possibly escalating to the role of file/driver maintainer. The plan is to systematically catalog the experiences and interactions to build a contributing journal. Also, as noted, the doctoral student maintains two supporting projects for the Linux ecosystem, kw and patch-hub, which are rich sources of insights.

4.1.2. Non-Participant Observation

Concerning the latter, the non-participant observation comprises three complementary fronts:

1. Mentor new contributors (newcomers) to the Linux ecosystem and collect data about their experience through cataloging mentors' perspectives and observations;
2. Conduct experiments with new contributors (newcomers) to the Linux ecosystem and, in a separate trial of experiments, veteran kernel developers to qualitatively and quantitatively measure how the FLOSS tools produced cover workflows and mitigate bottlenecks. For the newcomers' trials, questionnaires will be used to collect data, while for the veteran trials, data will be collected through telemetry using an anonymization approach similar to the one used by *Debian Popularity Contest*⁹;
3. Interact with key players from the Linux ecosystem and collect feedback through *in-loco* participation in community events, which was a suggestion given by the board of the Linux Foundation to our research group to the detriment of conducting surveys.

⁸[Rust for Linux project documentation.](#)

⁹[Debian Popularity Contest website.](#)

4.2. Multivocal Literature Review

A *Multivocal Literature Review* (MLR) is a systematic approach to synthesizing knowledge from diverse sources, including academic publications, Grey Literature (GL), practitioner reports, and other non-traditional texts. Essentially combining *Systematic Literature Review* (SLR) and *Grey Literature Review* (GLR), it acknowledges and integrates multiple perspectives and understandings of a research topic while identifying convergence, divergence, and gaps in the discourse [7]. *Wen (2021)* [19], in her master’s thesis, conducted an MLR to identify the gaps between state-of-the-art (academia understanding) and state-of-the-practice (practitioners understanding). As a secondary result, *Wen (2021)* [19] used the lessons learned when conducting the GLR to devise “guidelines to examine FLOSS projects through its community publications” due to the lack of a consolidated method to do GLR in this specific context. We plan to make an MLR similar to how *Wen (2021)* [19] did, but limiting the scope to development workflows and going back only 5 years since going back further would result in duplication of work.

The MLR will be meaningful for triangulating findings from the ethnographic case studies - our primary method - with state-of-the-art and state-of-the-practice to eliminate biases and strengthen the research.

5. Work Plan and Execution Schedule

The work plan is to apply the two research methods as concurrently as possible while continuously developing the mentioned FLOSS tools. We divided the research work into four phases, each one producing a new iteration of the taxonomy describing the workflows of the Linux kernel development model.

5.1. Phase I: Ethnographic Case Study I

In the first phase, the doctoral student immersed himself not so profoundly into the Linux ecosystem by exploring the dynamics of contributing to some parts of the ecosystem and mentoring graduate and undergraduate students as newcomers. In a sense, we dip ourselves into the ecosystem while observing how the movement of new developers into it works.

P1.1: Exploration into the Linux ecosystem. The doctoral student re-acclimated himself with the Linux kernel ecosystem by sending simple and exploratory patches to the

AMD GPU, IIO, and staging. He also recompiled the experiences of fellow researchers and members of the Linux community by reading their blog posts, some academic publications, and through direct conversations.

P1.2: Mentor newcomers I. In the first semester of 2024, the *Free Software Development* course ministered at IME-USP dedicated the first three-fourths of the discipline program to mentoring around 30 graduated and undergraduate students that involved teaching them how to: set up a testing environment; configure, compile, install, and boot a custom Linux kernel from source code; develop changes; send changes as contributions and participate in the review process; and much more. Essentially, lead them through all the steps described in the concept map of Figure 4. To accomplish this, students did in-person workshops based on tutorials devised by past researchers from the Software Systems Research Group of IME-USP, who are veteran Linux developers. The doctoral student was the principal mentor in this process. At the end of the course, students filled out a survey to collect data about their experiences.

P1.3: kw/patch-hub dev I. kw has been in development since 2019, and it already encoded many solutions for workflows at the start of 2024. In this first phase, the doctoral student invested great effort and time in developing, maintaining, and bringing new contributors to the project. From February to August, the doctoral student was also a mentor in *Google Summer of Code 2024* (GSoC 24) for two contributors in kw. The doctoral student transformed patch-hub - which was a proof-of-concept tool implemented in Bash script inside the kw codebase - into a dedicated project with its own codebase and written in Rust. During July, the doctoral student fully migrated the tool to the new project and surpassed its features, performance, and maintainability, which were unacceptable in the old form. Throughout the remainder of the second term of 2024, the project started to have recurring contributors forming a small community and really elevating it to a FLOSS project.

5.2. Phase II: Ethnographic Case Study II

The second phase continues the first phase, although the doctoral student deepens the immersion into the ecosystem by making more meaningful contributions and trying to be trusted with some maintainer responsibilities. Mentoring newcomers is virtually the same as in Phase I, but the researchers also conduct a first trial of experiments.

P2.1: Deep immersion into the Linux ecosystem. The doctoral student submits contributions that have a more profound impact on Linux subprojects and also engages in reviews sent from other contributors. The idea is to experience the role of non-casual contributors and, at least, experience one of the major responsibilities of maintainers, which is reviewing proposed changes. Additionally, the doctoral student will try to undertake formal responsibilities as a maintainer by organically and politely escalating his importance in the subproject(s).

P2.2: Mentor newcomers II. With another offering of the Free Software Development course in the first semester of 2025, mentoring newcomers will take the same shape, apart from natural adaptations and changes that resulted from the lessons learned in the first mentoring.

P2.3: Conduct Experiments I (newcomers). In the middle of the workshop series of **P2.2**, half of the course students will be randomly selected to use kw tools from that point onwards, while the other half will maintain use of the same tools used in the first mentoring. The overall program will remain the same, diverging from the tools used. Data describing the impact of including the use of kw tools will be collected through a survey, which will enable us to derive qualitative metrics, like how well bottleneck X of workflow Y was mitigated by kw Z tool(s), and also quantitative metrics like measure how much time workflow W demanded with and without the use of kw.

P2.4: kw/patch-hub dev II. With a better understanding of core workflows, in the second phase, the development of kw will focus on maintaining and refining the existing features it has while making a patch-hub with features enough to be included in a developer routine. The doctoral student is, once again, mentoring a contributor in GSoC 25. However, the work is focused on patch-hub.

5.3.Phase III: Unbiased Perspectives

Through ethnographic case studies, we can get valuable information from closely observing and immersing ourselves in the Linux development model. However, there are some innate biases regarding how the researchers (and their associates) view the ecosystem and the subset of the ecosystem considered. To mitigate this, triangulating these results with

academic and grey literature, along with the perspective of the community and its key players, is necessary.

P3.1: Conduct Multivocal Literature Review. Here, the researchers will cross the state-of-the-art from the last 5 years about Linux kernel development, focusing on workflows, with state-of-the-practice from community publications like blog posts, articles, and mailing lists.

P3.2: Collect feedback from the Linux community and its key players at events.

In previous research works done by past researchers from our research group, the board of the Linux Foundation requested that feedback from the community and key players be collected by participating in in-loco events and direct exchange instead of requesting surveys. Given this guidance, the researchers, especially the doctoral student, aim to participate in events such as *FOSDEM*¹⁰, as a speaker presenting the kw project and patch-hub, to interact with these sources of valuable information (which the student did on this DebConf 2025¹¹)

P3.3: kw/patch-hub dev III. Leveraging **Taxonomy I**, incorporate its knowledge in developing kw and patch-hub.

5.4. Phase IV: Final Validation.

From the coalition of the results from Phase I and II with Phase III, we aim to validate our reviewed theoretical framework describing the Linux kernel development model in a real-world scenario through experiments with veteran Linux developers.

P4.1: Conduct Experiments II (real practitioners). To capture the actual workflows of practitioners, this trial of experiments will be conducted by having veteran developers follow their day-to-day tasks naturally while using kw tools and patch-hub. Data will be generated seamlessly using kw, which already keeps statistics of some actions locally, like how many times the user compiled the kernel and the minimum, maximum, and average time it took for each day or period. Data telemetry is being investigated and planned to be implemented in the project soon. It will be inspired in the *Debian Popularity*

¹⁰[FOSDEM 2025 site.](#)

¹¹[DebConf 2025 schedule.](#)

Contest, where Debian users that comply send data about Debian packages to servers and go through a data anonymization process before analysis.

P4.2: kw/patch-hub dev IV. Leveraging **Taxonomy II**, incorporate its knowledge in developing kw and patch-hub.

5.5. Work Plan Overview

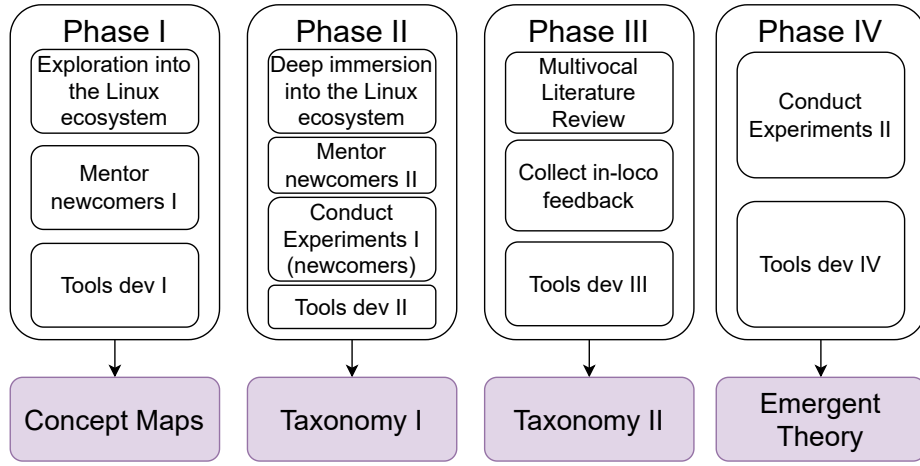


Figure 9: Research work plan.

Figure 9 is a diagram that summarizes the four phases. In this figure, each phase also describes the core work it represents (blue boxes at the middle of the figure), along with the iteration of the theoretical framework produced by the phase (magenta boxes at the bottom).

5.6. Execution Schedule

Given the tasks of each phase, Table 1 describes the five-year (60 months) schedule for this proposed doctoral research. **P0** refers to Phase 0 regarding the courses the doctoral student has to attend as part of his post-graduation program. For better visualization, activities in *italics and underlined* pertain to development tasks. Also, the cyan color represents the terms when the doctoral student plans to do a *Research Internship Abroad* (RIA) to Spain at the *Rey Juan Carlos University* (URJC), which is already agreed with the RIA advisor.

Table 1: 60 months schedule for the proposed doctoral research.

Activity	24.1	24.2	25.1	25.2	26.1	26.2	27.1	27.2	28.1	28.2
P0.0	X	X	X	X						
P1.1	X	X	X							
P1.2	X									
P1.3	X	X								
P2.1			X	X	X					
P2.2			X							
P2.3			X							
P2.4			X	X	X					
P3.1					X	X	X	X		
P3.2			X	X	X	X	X			
P3.3						X	X	X		
P4.1								X	X	
P4.2								X	X	X

6. Publications

In this section, we present the student’s publications. Below is a list of eight accepted papers where the student had some authorship role¹²:

- **D. Tadokoro**, R. Siqueira, and P. Meirelles, *Kworkflow: a Linux kernel Developer Automation Workflow System*. In *39th Brazilian Symposium on Software Engineering - Tools Track*.
- **D. Tadokoro** and P. Meirelles, *Can the Linux kernel sustain 30 more years of growth? Toward mitigating bottlenecks in its development model*. In *39th Brazilian Symposium on Software Engineering - Insightful Ideas and Emerging Results Track*.
- **D. Tadokoro**, R. Passos, and P. Meirelles, *Guidelines for Boosting Long-Lasting FLOSS Contributors*. In *DebConf 2025 - Academic Track*.
- **D. Tadokoro**, E. Mendes, and P. Meirelles, *Estudo, implementação e análise de estratégias para simplificar o modelo de contribuição do kernel Linux*. In *32º Simpósio Internacional de Iniciação Científica e Tecnológica da USP Universidade de São Paulo*.
- R. Passos, A. Pilone, **D. Tadokoro**, and P. Meirelles, *Streamlining Analysis on the Linux project with DUKS*. In *VISSOFT 2025 - Visualization Challenge*.
- L. Arcanjo, **D. Tadokoro**, and P. Meirelles, *ArKanho: a tool for detecting function-level Code Duplication in the Linux Kernel*. In *DebConf 2025 - Academic Track*.
- **D. Tadokoro** and P. Meirelles, *Estratégias de ensino para incentivar a participação consistente em projetos de Software Livre*. In *13th Workshop on Software Visualization, Evolution and Maintenance*.

¹²All accepted publications are directly related to or stem from the research work presented in this proposal, even those where the student is on the record as a co-author.

- R. Passos, A. Pilone, **D. Tadokoro**, and P. Meirelles, *DUKS: visualizações e análises unificadas para o Kernel Linux*. In *13th Workshop on Software Visualization, Evolution and Maintenance*.

Beyond accepted publications, there are two works in progress to be submitted to the *International Conference on Software Engineering 2026 - Software Engineering Education and Training (ICSE 2026 - SEET)* and the journal *Information and Software Technology (IST)*.

References

- [1] Jonathan Corbet. *Finding real-world kernel subsystems*. 2021. URL: <https://lwn.net/Articles/844539/> (visited on 03/20/2025).
- [2] Jonathan Corbet. *How 4.4’s patches got to the mainline*. 2014. URL: <https://lwn.net/Articles/670209/> (visited on 03/20/2025).
- [3] Jonathan Corbet. *Patch flow into the mainline for 4.14*. 2017. URL: <https://lwn.net/Articles/737093/> (visited on 03/20/2025).
- [4] Jonathan Corbet and Greg Kroah-Hartman. *Linux Kernel Development Report*. 2017. URL: <https://www.linuxfoundation.org/resources/publications/linux-kernel-report-2017?hsLang=en> (visited on 03/20/2025).
- [5] W. Edmonds and Thomas Kennedy. *An Applied Guide to Research Designs: Quantitative, Qualitative, and Mixed Methods*. Jan. 2017. ISBN: 9781483317274.
- [6] Karl Fogel. *Producing open source software: How to run a successful free software project*. ”O’Reilly Media, Inc.”, 2017.
- [7] Vahid Garuslu, Michael Felderer, and Mika Mäntylä. “Guidelines for including grey literature and conducting multivocal literature reviews in software engineering”. In: *Information and Software Technology* 106 (Sept. 2018).
- [8] Barney Glaser and Anselm Strauss. *Discovery of grounded theory: Strategies for qualitative research*. Routledge, 2017.
- [9] “Governing open source software through coordination processes”. In: *Information and Organization* 27.2 (2017), pp. 116–135.
- [10] Daniel Izquierdo-Cortazar et al. “Using Metrics to Track Code Review Performance”. In: *Proceedings of the 21st International Conference on Evaluation and Assessment in Software Engineering*. EASE ’17. Karlskrona, Sweden: Association for Computing Machinery, 2017, pp. 214–223. ISBN: 9781450348041.

- [11] Greg Kroah-Hartman. *Kernel Recipes 2016 - Patches carved into stone tablets*. Licensed under CC BY-NC-SA 4.0. Image modified by adding the overlaying rectangle with the “contributor-maintainer interface” text. Linux Kernel community. 2016. URL: <https://github.com/gregkh/presentation-stone-tools/tree/master> (visited on 05/21/2025).
- [12] Leonardo Alexandre Ferreira Leite, Paulo Roberto Miranda Meirelles, and Fábio Kon. “A grounded theory of organizational structures for development and infrastructure professionals in software-producing organizations”. MA thesis. Universidade de São Paulo, 2022.
- [13] A. Lindberg, Xuan Xiao, and Kalle Lyytinen. “Theorizing Modes of Open Source Software Development”. In: *2014 47th Hawaii International Conference on System Sciences*. 2014, pp. 4568–4577.
- [14] Nicolas Palix et al. “Faults in Linux: Ten Years Later”. In: *ACM SIGARCH Computer Architecture News* 39 (Mar. 2011).
- [15] Eric Raymond. “The cathedral and the bazaar”. In: *Knowledge, Technology & Policy* 12.3 (1999), pp. 23–49.
- [16] Peter C. Rigby et al. “Peer Review on Open-Source Software Projects: Parameters, Statistical Models, and Theory”. In: 23.4 (Sept. 2014).
- [17] Abraham Silberschatz, James L Peterson, and Peter B Galvin. *Operating System Concepts*. Addison-Wesley Longman Publishing Co., Inc., 2008.
- [18] Linus Torvalds. *Linux 4.19-rc4 released, an apology, and a maintainership note*. 2018. URL: <https://lkml.org/lkml/2018/9/16/167> (visited on 03/20/2025).
- [19] Melissa Shihfan Ribeiro Wen. “What Happens When the Bazaar Grows: A comprehensive study on the contemporary Linux kernel development model”. MA thesis. Universidade de São Paulo, 2021.